# 8051 Microcontroller

## I/O Ports, Reset, and Timing (Lecture 8)

## 1. I/O Ports

- The 8051 has 40 pins.
- 32 pins are used as I/O pins.
- Each port is 8 pins wide, so 32 / 8 = 4 ports.
- The 8051 therefore has 4 I/O ports (or 4 × 8 = 32 I/O pins).

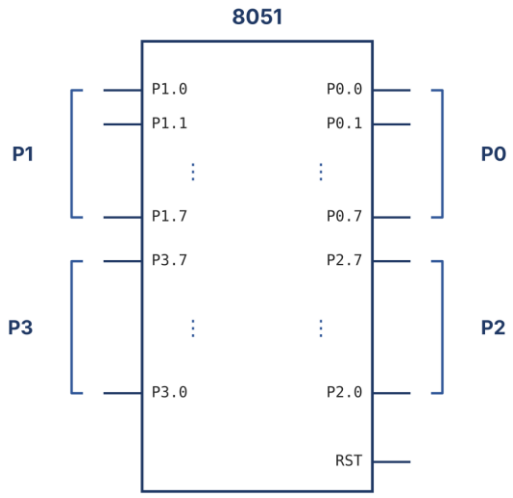



*Figure 1. The four I/O ports of the 8051 (simplified; the pin order follows the lecture sketch).*

**All ports (I/O pins) can be used as input / output.**

### 1.1 Port 0 (P0)

- It must be connected to pull-up resistors (10 kΩ).
- It is used to connect the 8051 with external memory.

  It carries the low part of the addresses: P0 ↔ AD0 – AD7.

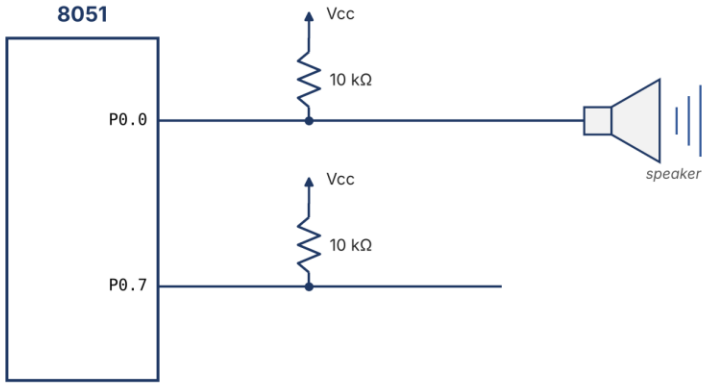



*Figure 2. P0 pins with 10 kΩ pull-up resistors to Vcc; P0.0 drives a speaker.*

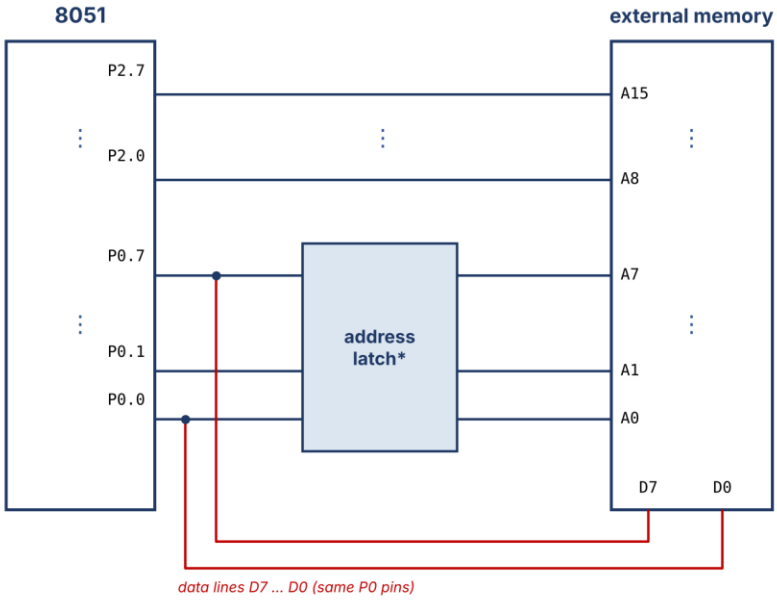



*Figure 3. Connecting external memory: P2 supplies A8–A15, P0 supplies A0–A7 (through a latch) and the data lines.*

** The box between P0 and the low address pins is the address latch that holds the low address byte while P0 is reused for data.*

## 1.2 Port 1 (P1)

- It is used only as input / output (no additional task).

## 1.3 Port 2 (P2)

- It is used to connect external memory.
- It carries the high part of the addresses (A8 – A15).

## 1.4 Port 3 (P3)

- Each pin has a special task (see slide #7).
- P3.6 → WR (write signal, active low) } external memory

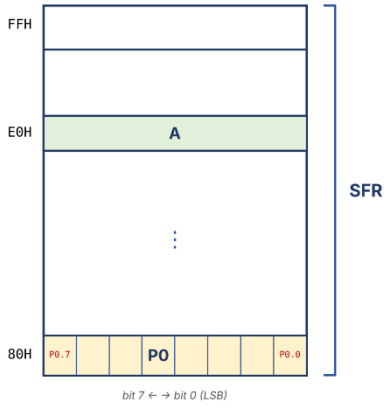
- P3.7 → RD (read signal, active low) } external memory
- Other control pins of the external-memory interface: EA, PSEN, ALE (see slide #6).

| Port | Besides I/O, it is used for | Notes |
|---|---|---|
| **P0** | External memory: low address byte (AD0–AD7) | Needs 10 kΩ pull-ups |
| **P1** | Nothing, I/O only | |
| **P2** | External memory: high address byte (A8–A15) | |
| **P3** | Special task for each pin (e.g., P3.6 = WR, P3.7 = RD) | See slide #7 |

## 2. The Reset Pin (RST)

RST is the reset pin. It is used to terminate all activities and start with initial values (upon reset). The initial values are:

| Register | Value after reset |
|---|---|
| A, B, R0 → R7 | 00H |
| PC, DPTR | 0000H |
| SP | 07H |
| P0, P1, P2, P3 | FFH |




## 3. Configuring a Port as Input or Output

- By default, all ports are input ports (they hold FFH after reset).
- The programmer can make any I/O pin an input or an output.
- **To make any pin an input, send 1 to it.**
- **To make any pin an output, send 0 to it.**

*Figure 4. The ports and the accumulator live in the SFR area: A is at E0H and P0 is at 80H (P0.0 is the least significant bit).*

### 3.1 Examples

**Configure P3 as input:**

```
        MOV   P3, #11111111B
        MOV   P3, #0FFH          ; same thing in hex
```

**Configure single pins:**

```
        SETB  P1.0               ; P1.0 is an input pin
        CLR   P1.2               ; P1.2 is an output pin
```

**Configure P2 to be an output port:**

```
        MOV   P2, #00H      ; or MOV P2,#0
```

## 4. Controlling an LED from a Port Pin

The same pin can turn an LED on or off, but the logic level that lights the LED depends on how the LED is wired.

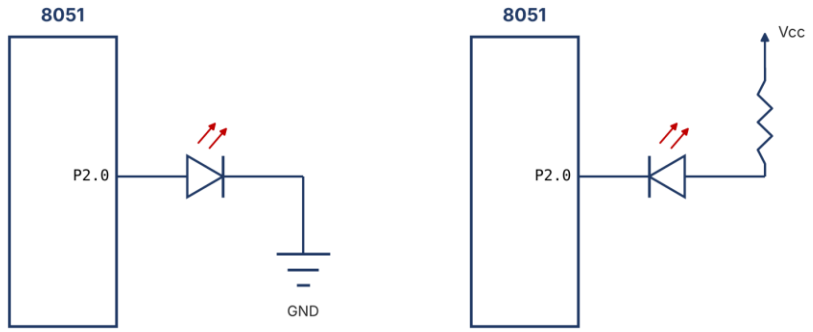



*Figure 5. Left: LED between P2.0 and ground (active high). Right: LED between Vcc (through a resistor) and P2.0 (active low).*

| Circuit | Turn the LED ON | Turn the LED OFF |
|---|---|---|
| LED between P2.0 and Gnd | `SETB P2.0` | `CLR P2.0` |
| LED between Vcc and P2.0 | `CLR P2.0` | `SETB P2.0` |

# 5. Example Programs

## 5.1 Write a code to read P0 and send its 2's complement to P1 continuously

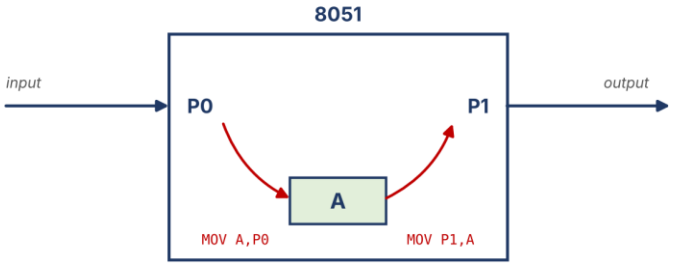



*Figure 6. Data path: P0 → A → P1.*

```
        ORG   0000H
        MOV   P0, #0FFH      ; P0 as input
Back:   MOV   A, P0
        CPL   A              ; 1's complement
        INC   A              ; +1  ->  2's complement
        MOV   P1, A
        SJMP  Back
```

## 5.2 Toggle P1 and P2 continuously

Toggling a pin means complementing it (1 becomes 0 and 0 becomes 1).

**Solution 1 — using CPL:**

```
Back:   MOV   A, P1
        CPL   A
        MOV   P1, A
        MOV   A, P2
        CPL   A
        MOV   P2, A
        ACALL DELAY
        SJMP  Back

DELAY:  MOV   R2, #200
HERE:   DJNZ  R2, HERE
        RET
```

**Solution 2 — without CPL:**

```
Back:   MOV   A, #55H
        MOV   P1, A
        MOV   P2, A
        ACALL DELAY
        MOV   A, #0AAH
        MOV   P1, A
        MOV   P2, A
        ACALL DELAY
        SJMP  Back
```

| Value | Binary (bit 7 → bit 0) | Pins |
|---|---|---|
| 55H | 0 1 0 1 0 1 0 1 | 0 = off, 1 = on, alternating |
| AAH | 1 0 1 0 1 0 1 0 | the opposite pattern (the complement of 55H) |

## 5.3 Write a program to create a square wave with 50% duty cycle on P1.4

Duty cycle = Time of 1 / T  (Time 1 = the time the signal is high, T = the period)

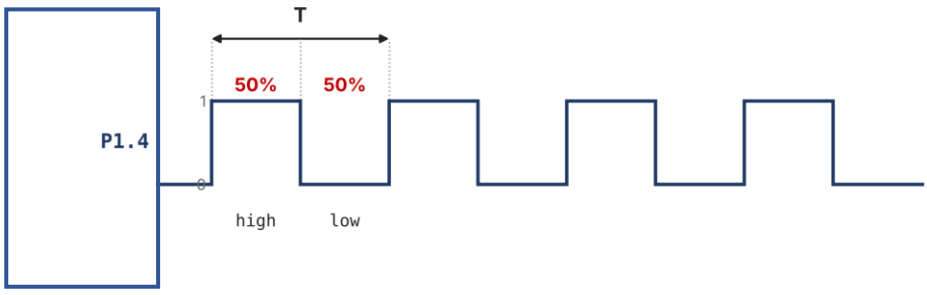



*Figure 7. 50% duty cycle: high for half of the period, low for the other half.*

```
        ORG   0
Back:   SETB  P1.4
        ACALL DELAY
        CLR   P1.4
        ACALL DELAY
        SJMP  Back

DELAY:  MOV   R4, #250
AGAIN:  NOP
        DJNZ  R4, AGAIN
        RET
```

**NOP:** No operation (do nothing). It only updates the PC.

## 5.4 Write a code to create a wave with 75% duty cycle on P2.7

Let **x** be the time the pin is high and **y** the time it is low (one DELAY). Then x / y = 75 / 25 = 3, so x = 3y: keep the pin high for three delays and low for one.

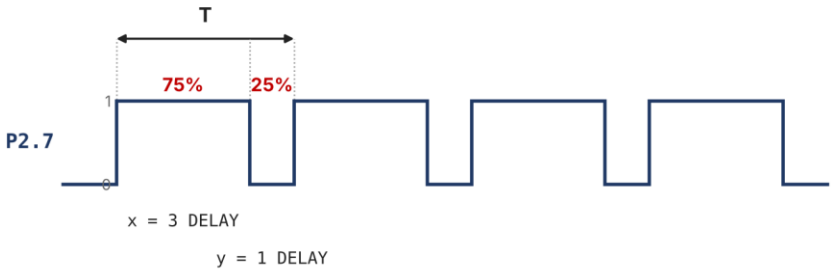



*Figure 8. 75% duty cycle: high for three DELAY periods (x), low for one (y).*

```
        ORG   0
Back:   SETB  P2.7
        ACALL DELAY
        ACALL DELAY
        ACALL DELAY
        CLR   P2.7
        ACALL DELAY
        SJMP  Back
```

The DELAY subroutine is not repeated in the notes; use the one from the previous example.

**Another case — 66% duty cycle:** x / y = 66 / 34 ≈ 2, so keep the pin high for about two delays and low for one.

# 6. Clock, Machine Cycle, and Delay Time

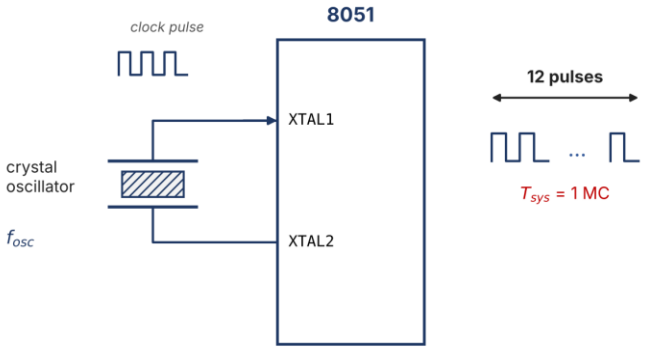



*Figure 9. The crystal oscillator drives XTAL1/XTAL2; the 8051 counts 12 clock pulses per machine cycle.*

- The 8051 divides the crystal frequency by 12:

$F_{sys} = f_{osc} / 12$ or $f_{sys} = f_{XTAL} / 12$

- The machine cycle (MC) is the system period:

$MC = 12 / f_{osc}$ => $T_{sys} = MC = 12 / f_{osc}$

## 6.1 Example: $f_{osc}$ = 11.0592 MHz

```
f_sys = 11.0592 MHz / 12 = 0.9216 MHz
MC    = 12 / 11.0592 MHz = 1.085 µs
```

## 6.2 How long does an instruction take?

Instructions take 1, 2, or 4 machine cycles:

| Instruction length | Oscillator cycles |
|---|---|
| 1 MC | 12 cycles |
| 2 MC | 24 cycles |
| 4 MC | 4 × 12 = 48 cycles |

## 6.3 Example: execution time of a code fragment (24 MHz crystal)

Assume the crystal oscillator is 24 MHz. Calculate the time of the following code.

| Instruction | MC |
|---|---|
| MOV A, R4 | 1 |
| NOP | 1 |
| NOP | 1 |
| MUL AB | 4 |
| MOV R2, B | 2 |

```
MC   = 12 / f_osc = 12 / 24 MHz = 0.5 µs
Time = (1 + 1 + 1 + 4 + 2) x MC
     = 9 x 0.5 µs = 4.5 µs
```

## 6.4 Example: delay of a loop (6 MHz crystal)

Assume XTAL = 6 MHz. Compute the time delay of this subroutine.

| | MC | Times executed | MC total |
|---|---|---|---|
| DELAY: MOV R2, #100 | 1 | 1 | 1 × 1 = 1 |
| NOP | 1 | 1 | 1 × 1 = 1 |
| HERE: DJNZ R2, HERE | 2 | 100 | 2 × 100 = 200 |
| RET | 2 | 1 | 2 × 1 = 2 |

```
MC         = 12 / 6 MHz = 2 µs
Time delay = (1 + 1 + 200 + 2) x MC
           = 204 x 2 µs = 408 µs
```

## 6.5 Example

Given $f_{XTAL}$ = 12 MHz, the machine cycle is:

```
MC = 12 / fXTAL = 12 / 12 MHz = 1 µs
```

The delay subroutine for this question is the nested loop below.

```
DELAY:  CLR   A
        MOV   R2, #4
ME2:    MOV   R1, #50
ME:     MOV   P1, A
        CLR   A
        NOP
        NOP
        DJNZ  R1, ME
        DJNZ  R2, ME2
        RET
```

©2026 Dr. Fadi Alzhouri

The inner loop runs 50 times, and the outer loop runs 4 times, so the inner body is executed 50 × 4 = 200 times.

| | MC | Times executed | MC total |
|---|---|---|---|
| CLR A | 1 | 1 | 1 |
| MOV R2, #4 | 1 | 1 | 1 |
| MOV R1, #50 | 1 | 4 | 4 |
| MOV P1, A | 1 | 200 | 200 |
| CLR A | 1 | 200 | 200 |
| NOP | 1 | 200 | 200 |
| NOP | 1 | 200 | 200 |
| DJNZ R1, ME | 2 | 200 | 400 |
| DJNZ R2, ME2 | 2 | 4 | 8 |
| RET | 2 | 1 | 2 |
| **Total** | | | |

```
Delay = (1 + 1 + 4 + 200 + 200 + 200 + 200 + 400 + 8 + 2) x MC
      = 1216 x 1 µs = 1216 µs  (about 1.21 ms)
```